

Dht11+lcd

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <DHT.h>

#define DHTPIN 12
#define DHTTYPE DHT11

DHT dht(DHTPIN, DHTTYPE);
LiquidCrystal_I2C lcd(0x27, 16, 2);

void setup() {
  dht.begin();

  lcd.init();
  lcd.backlight();

  lcd.setCursor(0, 0);
  lcd.print("DHT11 Sensor");
  delay(2000);
}

void loop() {
  float temperature = dht.readTemperature();
  float humidity = dht.readHumidity();

  if (isnan(temperature) || isnan(humidity)) {
    lcd.clear();
    lcd.setCursor(0, 0);
    lcd.print("Sensor Error");
    delay(2000);
    return;
  }

  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("Temp: ");
  lcd.print(temperature, 1);
  lcd.print((char)223);
  lcd.print("C");

  lcd.setCursor(0, 1);
  lcd.print("Humidity: ");
  lcd.print(humidity, 1);
  lcd.print("%");

  delay(2000);
}
```

servo+ultrasonic

```
#include <Wire.h>
#include <LiquidCrystal_I2C.h>
#include <Servo.h>

#define TRIG 5
#define ECHO 6
```

```

#define SERVO_PIN 9

Servo s;
LiquidCrystal_I2C lcd(0x27, 16, 2);

long getDistance() {
    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);

    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);

    digitalWrite(TRIG, LOW);

    long duration = pulseIn(ECHO, HIGH, 30000);

    if (duration == 0)
        return 400;

    return duration * 0.0343 / 2;
}

void setup() {
    pinMode(TRIG, OUTPUT);
    pinMode(ECHO, INPUT);

    s.attach(SERVO_PIN);

    // Serial communication with Python
    Serial.begin(9600);

    lcd.init();
    lcd.backlight();

    lcd.setCursor(0, 0);
    lcd.print("Radar Starting");

    delay(1000);
    lcd.clear();
}

void loop() {

    // 0 -> 180
    for (int angle = 0; angle <= 180; angle += 2) {

        s.write(angle);
        delay(20);

        long distance = getDistance();

        // Send data to Python
        Serial.print(angle);
        Serial.print(",");
        Serial.println(distance);

        // LCD

```

```

    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Angle: ");
    lcd.print(angle);
    lcd.print((char)223);

    lcd.setCursor(0, 1);
    lcd.print("Dist: ");
    lcd.print(distance);
    lcd.print(" cm");

    delay(50);
}

// 180 -> 0
for (int angle = 180; angle >= 0; angle -= 2) {

    s.write(angle);
    delay(20);

    long distance = getDistance();

    // Send data to Python
    Serial.print(angle);
    Serial.print(",");
    Serial.println(distance);

    // LCD
    lcd.clear();

    lcd.setCursor(0, 0);
    lcd.print("Angle: ");
    lcd.print(angle);
    lcd.print((char)223);

    lcd.setCursor(0, 1);
    lcd.print("Dist: ");
    lcd.print(distance);
    lcd.print(" cm");

    delay(50);
}
}

```

python code

```

# Create project
mkdir ~/radar-project
cd ~/radar-project

# Create virtual environment
python3 -m venv radar-env

# Activate environment
source radar-env/bin/activate

# Install libraries

```

```
python -m pip install --upgrade pip
python -m pip install pyserial pygame-ce
```

```
# Verify
python -c "import serial, pygame; print('Setup successful')"
```

data flow

```
HC-SR04
  ↓
Arduino
  ↓
USB Serial
  ↓
PySerial
  ↓
Python
  ↓
Pygame
  ↓
Sonar Display
```

```
import pygame
import serial
import math
import time
```

```
# =====
# SERIAL CONFIGURATION
# =====
```

```
PORT = "/dev/cu.usbserial-A5069RR4"
BAUD = 9600
```

```
ser = serial.Serial(
    PORT,
    BAUD,
    timeout=0
)
```

```
# Arduino resets when the serial port opens
time.sleep(2)
```

```
# Remove old data
ser.reset_input_buffer()
```

```
# =====
# PYGAME INITIALIZATION
# =====
```

```
pygame.init()
```

```
WIDTH = 1200
HEIGHT = 800
```

```
screen = pygame.display.set_mode((WIDTH, HEIGHT))
pygame.display.set_caption("YANTRIKA - ULTRASONIC SONAR")
```

```

clock = pygame.time.Clock()

# =====
# COLORS
# =====

BLACK = (2, 8, 5)

DARK_GREEN = (0, 35, 15)
GRID_GREEN = (0, 90, 35)

GREEN = (0, 180, 70)
BRIGHT_GREEN = (0, 255, 100)

WHITE_GREEN = (180, 255, 210)

# =====
# RADAR SETTINGS
# =====

CENTER_X = WIDTH // 2
CENTER_Y = HEIGHT - 80

CENTER = (CENTER_X, CENTER_Y)

RADAR_RADIUS = 650

MAX_DISTANCE = 100

# =====
# CURRENT SENSOR VALUES
# =====

current_angle = 90.0
current_distance = 100.0

# =====
# SERIAL BUFFER
# =====

serial_buffer = ""

# =====
# SONAR ECHO STORAGE
# =====

# Each echo:
#
# (angle, distance, timestamp)

echoes = []

```

```
ECHO_LIFETIME = 2.0
```

```
# =====  
# FONTS  
# =====
```

```
font_large = pygame.font.Font(None, 38)  
font_medium = pygame.font.Font(None, 28)  
font_small = pygame.font.Font(None, 22)
```

```
# =====  
# POLAR COORDINATES → SCREEN  
# =====
```

```
def polar_to_screen(angle, distance):
```

```
    radius = (  
        distance / MAX_DISTANCE  
    ) * RADAR_RADIUS
```

```
    radians = math.radians(180 - angle)
```

```
    x = (  
        CENTER_X  
        + radius * math.cos(radians)  
    )
```

```
    y = (  
        CENTER_Y  
        - radius * math.sin(radians)  
    )
```

```
    return int(x), int(y)
```

```
# =====  
# DRAW RADAR ARC  
# =====
```

```
def draw_arc(radius):
```

```
    pygame.draw.arc(  
        screen,  
        GRID_GREEN,  
        (  
            CENTER_X - radius,  
            CENTER_Y - radius,  
            radius * 2,  
            radius * 2  
        ),  
        0,  
        math.pi,  
        2  
    )
```

```

# =====
# DRAW RADAR GRID
# =====

def draw_grid():

    # Distance rings

    for distance in [20, 40, 60, 80, 100]:

        radius = (
            distance / MAX_DISTANCE
        ) * RADAR_RADIUS

        draw_arc(radius)

    # Angle lines

    for angle in range(0, 181, 30):

        radians = math.radians(180 - angle)

        x = (
            CENTER_X
            + RADAR_RADIUS * math.cos(radians)
        )

        y = (
            CENTER_Y
            - RADAR_RADIUS * math.sin(radians)
        )

        pygame.draw.line(
            screen,
            DARK_GREEN,
            CENTER,
            (int(x), int(y)),
            1
        )

    # Horizontal baseline

    pygame.draw.line(
        screen,
        GRID_GREEN,
        (
            CENTER_X - RADAR_RADIUS,
            CENTER_Y
        ),
        (
            CENTER_X + RADAR_RADIUS,
            CENTER_Y
        ),
        2
    )

```

```

# =====
# DISTANCE LABELS
# =====

def draw_distance_labels():

    for distance in [20, 40, 60, 80, 100]:

        radius = (
            distance / MAX_DISTANCE
        ) * RADAR_RADIUS

        text = font_small.render(
            f"{distance} cm",
            True,
            GRID_GREEN
        )

        screen.blit(
            text,
            (
                CENTER_X + 8,
                CENTER_Y - radius - 12
            )
        )

# =====
# ANGLE LABELS
# =====

def draw_angle_labels():

    for angle in [0, 30, 60, 90, 120, 150, 180]:

        x, y = polar_to_screen(
            angle,
            MAX_DISTANCE
        )

        text = font_small.render(
            f"{angle}°",
            True,
            GRID_GREEN
        )

        screen.blit(
            text,
            (
                x - 15,
                y - 10
            )
        )

```

```

# =====
# DRAW SWEEP
# =====

def draw_sweep(angle):

    beam = pygame.Surface(
        (WIDTH, HEIGHT),
        pygame.SRCALPHA
    )

    radians = math.radians(180 - angle)

    x = (
        CENTER_X
        + RADAR_RADIUS * math.cos(radians)
    )

    y = (
        CENTER_Y
        - RADAR_RADIUS * math.sin(radians)
    )

    # Wide transparent beam

    pygame.draw.line(
        beam,
        (0, 255, 100, 40),
        CENTER,
        (int(x), int(y)),
        18
    )

    # Bright sweep line

    pygame.draw.line(
        beam,
        (0, 255, 120, 230),
        CENTER,
        (int(x), int(y)),
        3
    )

    screen.blit(
        beam,
        (0, 0)
    )

# =====
# DRAW ECHOES
# =====

def draw_echoes():

    now = time.time()

```

```

for angle, distance, timestamp in echoes:

    age = now - timestamp

    if age >= ECHO_LIFETIME:
        continue

    # Fade with time

    intensity = 1.0 - (
        age / ECHO_LIFETIME
    )

    brightness = max(
        30,
        int(255 * intensity)
    )

    x, y = polar_to_screen(
        angle,
        distance
    )

    # Glow

    pygame.draw.circle(
        screen,
        (
            0,
            brightness // 2,
            brightness // 4
        ),
        (x, y),
        12
    )

    # Target

    pygame.draw.circle(
        screen,
        (
            0,
            brightness,
            brightness // 2
        ),
        (x, y),
        5
    )

# =====
# READ ARDUINO SERIAL DATA
# =====

def read_serial():

    global current_angle

```

```

global current_distance
global serial_buffer

# Read all currently available bytes

while ser.in_waiting:

    data = ser.read(
        ser.in_waiting
    ).decode(
        "utf-8",
        errors="ignore"
    )

    serial_buffer += data

# Only process complete lines

while "\n" in serial_buffer:

    line, serial_buffer = serial_buffer.split(
        "\n",
        1
    )

    line = line.strip()

    if not line:
        continue

    # Expected format:
    #
    # 90,35

    parts = line.split(",")

    if len(parts) != 2:
        continue

    try:

        new_angle = float(parts[0])
        new_distance = float(parts[1])

    except ValueError:

        continue

    # -----
    # VALIDATE ANGLE
    # -----

    if new_angle < 0 or new_angle > 180:
        continue

    # -----
    # VALIDATE DISTANCE

```

```

# -----

if new_distance <= 0 or new_distance > 400:
    continue

# -----
# ACCEPT READING
# -----

current_angle = new_angle
current_distance = new_distance

# Store sonar echo

if new_distance <= MAX_DISTANCE:

    echoes.append(
        (
            new_angle,
            new_distance,
            time.time()
        )
    )

# =====
# REMOVE OLD ECHOES
# =====

def clean_echoes():

    global echoes

    now = time.time()

    echoes = [
        echo
        for echo in echoes
        if now - echo[2] < ECHO_LIFETIME
    ]

# =====
# HUD
# =====

def draw_hud():

    # Title

    title = font_large.render(
        "YANTRIKA // ULTRASONIC SONAR",
        True,
        BRIGHT_GREEN
    )

    screen.blit(

```

```

        title,
        (30, 25)
    )

    # Angle

    angle_text = font_medium.render(
        f"ANGLE      {current_angle:6.1f}°",
        True,
        WHITE_GREEN
    )

    screen.blit(
        angle_text,
        (30, 70)
    )

    # Distance

    if current_distance <= MAX_DISTANCE:

        distance_text = font_medium.render(
            f"DISTANCE  {current_distance:6.1f} cm",
            True,
            WHITE_GREEN
        )

    else:

        distance_text = font_medium.render(
            "DISTANCE  OUT OF RANGE",
            True,
            GRID_GREEN
        )

    screen.blit(
        distance_text,
        (30, 105)
    )

    # System information

    status_text = font_small.render(
        "HC-SR04  |  SG90  |  9600 BAUD",
        True,
        GRID_GREEN
    )

    screen.blit(
        status_text,
        (30, 140)
    )

# =====
# MAIN LOOP
# =====

```

```

running = True

while running:

    # -----
    # WINDOW EVENTS
    # -----

    for event in pygame.event.get():

        if event.type == pygame.QUIT:

            running = False

    # -----
    # READ SERIAL
    # -----

    read_serial()

    # -----
    # CLEAN OLD ECHOES
    # -----

    clean_echoes()

    # -----
    # BACKGROUND
    # -----

    screen.fill(BLACK)

    # -----
    # RADAR
    # -----

    draw_grid()

    draw_distance_labels()

    draw_angle_labels()

    # -----
    # SONAR ECHOES
    # -----

    draw_echoes()

    # -----
    # SWEEP
    # -----

    draw_sweep(current_angle)

    # -----
    # CENTER

```

```

# -----

pygame.draw.circle(
    screen,
    BRIGHT_GREEN,
    CENTER,
    7
)

pygame.draw.circle(
    screen,
    GREEN,
    CENTER,
    14,
    2
)

# -----
# HUD
# -----

draw_hud()

# -----
# DISPLAY
# -----

pygame.display.flip()

clock.tick(60)

# =====
# CLEANUP
# =====

ser.close()

pygame.quit()

```